

Clase 080 — Cracking de contraseñas con John y Hashcat

Parte: **3 — Hacking ético y pentesting: metodología** · Fuente: *Documentación oficial de Hashcat y John the Ripper (Openwall); NIST SP 800-63B* 🕒 Duración estimada: **130 min** · Nivel: **Avanzado**

Objetivo

El alumno aprenderá a recuperar contraseñas en texto claro a partir de hashes obtenidos legítimamente en un laboratorio, usando John the Ripper y Hashcat: identificar el tipo de hash correcto, elegir entre ataque de diccionario, reglas de mutación, máscara o híbrido, aprovechar la aceleración por GPU, y a partir de los resultados evaluar la fortaleza real de la política de contraseñas de una organización.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. Identificar correctamente el tipo de hash y su modo correspondiente en Hashcat (`-m`) y John (`--format`).
2. Ejecutar ataques de diccionario, de reglas, de máscara e híbridos según el contexto del hash.
3. Configurar y aprovechar la aceleración por GPU en Hashcat frente al uso de CPU.
4. Aplicar reglas de mutación conocidas (best64, rockyou-30000) sobre wordlists estándar como rockyou.txt.
5. Explicar por qué los KDF lentos (bcrypt, scrypt, Argon2) resisten el cracking masivo por diseño.
6. Redactar una recomendación de política de contraseñas basada en los resultados obtenidos.

Temas

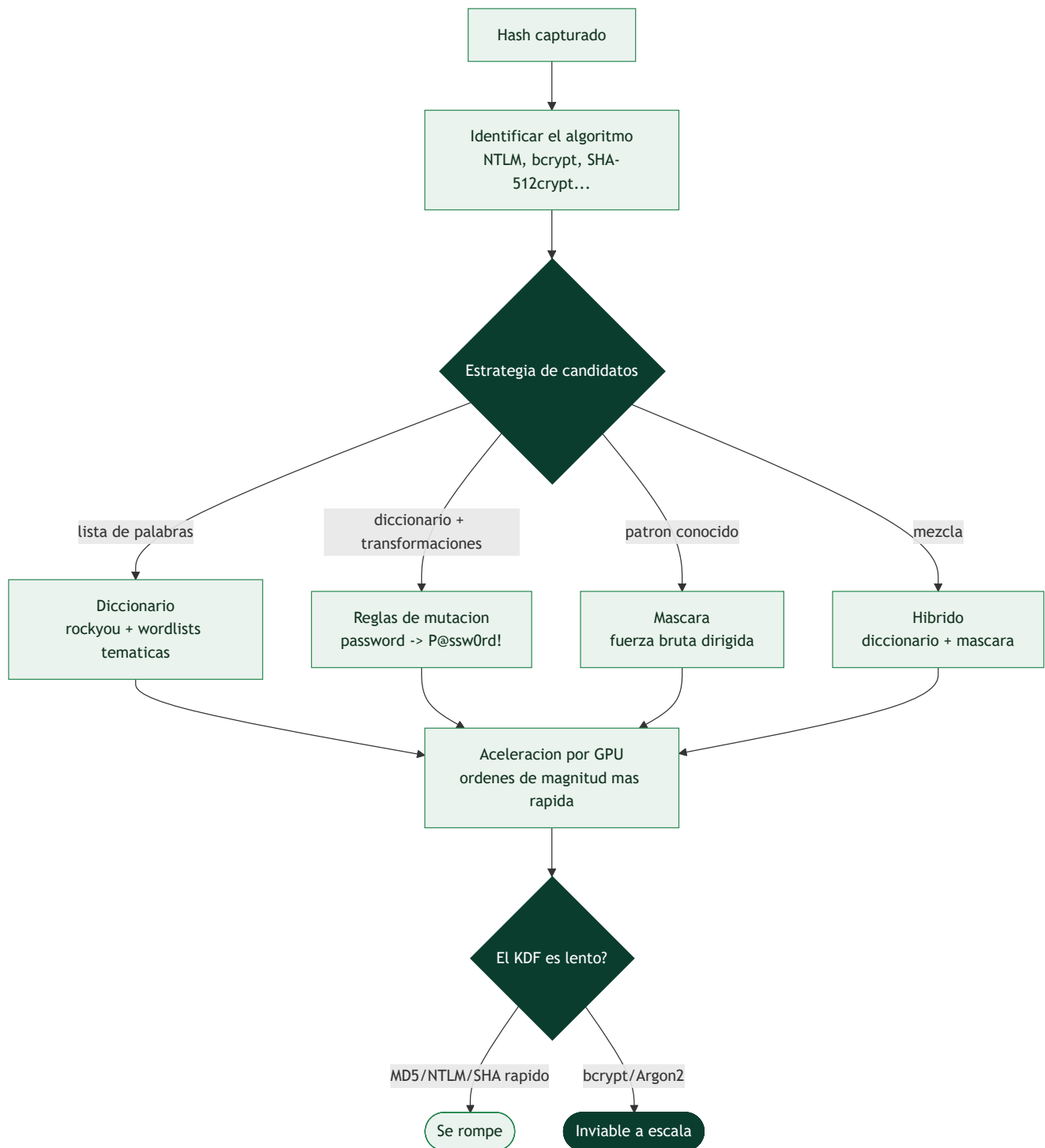
#	Tema	Por qué importa
1	Identificación del tipo de hash	Un modo incorrecto en Hashcat/John no produce ningún resultado
2	Ataque de diccionario	El método más eficiente contra contraseñas elegidas por humanos
3	Reglas de mutación	Transforman <code>password</code> en variantes como <code>P@ssw0rd!</code> automáticamente
4	Ataques de máscara	Fuerza bruta dirigida por un patrón conocido, mucho más eficiente que la fuerza bruta pura
5	Ataques híbridos	Combinan diccionario y máscara para cubrir patrones parcialmente predecibles
6	Aceleración por GPU	Diferencia de varios órdenes de magnitud frente al cracking por CPU
7	Salting y rainbow tables	El salt individual por hash inutiliza las tablas precomputadas
8	KDF lentos (bcrypt/Argon2)	Diseñados deliberadamente para hacer inviable el cracking a escala

Explicación en profundidad

Los hashes robados solo valen si se rompen (o se reutilizan)

La post-explotación y la enumeración producen **hashes** de contraseñas. Algunos se reutilizan directamente (Pass-the-Hash, [Clase 078](#)), pero muchos —los de servicios web, los de aplicaciones, los que hay que usar en un portal— necesitan **la contraseña en claro**, y recuperarla es el **cracking**. La idea es la de la [Clase 057](#) vista desde el lado ofensivo: como un hash no se invierte, se **hashean candidatos** hasta encontrar uno que coincida. Todo el arte está en generar buenos candidatos rápido.

El primer paso, y el que más se falla, es **identificar el tipo de hash**. John the Ripper y Hashcat necesitan saber el algoritmo exacto (NTLM, MD5, bcrypt, SHA-512crypt...) para elegir el modo correcto; un modo equivocado no produce **ningún** resultado, y el pentester novato pasa horas atacando con el algoritmo incorrecto. Herramientas como `hashid` o `hash-identifier` ayudan, pero el contexto (de dónde salió el hash) suele ser la mejor pista.



Diccionario, máscara, e híbrido: de lo probable a lo exhaustivo

Los humanos eligen contraseñas predecibles, y la estrategia de cracking lo explota en orden de eficiencia. El **ataque de diccionario** prueba listas de contraseñas conocidas — **rockyou** es la clásica, con millones de contraseñas reales filtradas— y resuelve la mayoría de los casos porque la gente reutiliza las mismas. Las **reglas de mutación** multiplican ese diccionario aplicando transformaciones que imitan cómo la gente "fortalece" sus contraseñas: `password` → `Password1`, `P@ssw0rd!`, `password2024`. Los **ataques de máscara** son fuerza bruta **dirigida por un patrón** conocido (`?u?l?l?l?l?d?d` = mayúscula, cinco minúsculas, dos dígitos): al acotar el espacio, son mucho más rápidos que la fuerza bruta ciega. Y los **ataques híbridos** combinan diccionario y máscara (una palabra del diccionario seguida de dos dígitos), cubriendo el patrón humano más común de todos.

GPU y KDF: la carrera armamentística

La **aceleración por GPU** es lo que hace el cracking viable: una tarjeta gráfica prueba miles de millones de candidatos por segundo contra un hash rápido, órdenes de magnitud por encima de una CPU. Contra MD5, SHA-1, NTLM o SHA-256 —hashes **rápidos**, pensados para velocidad— esto es demoledor: contraseñas mediocres caen en segundos. Aquí es donde se cierra el círculo con la Parte 2: los **KDF lentos y memory-hard** (bcrypt, scrypt, Argon2) están diseñados precisamente para que **esta** aceleración no sirva, encareciendo cada intento hasta hacer el ataque inviable a escala. Y el **salt** individual por hash (la [Clase 057](#)) inutiliza las **rainbow tables**, obligando a atacar cada hash por separado.

El valor de esta clase para el pentest es doble. Por un lado, recuperar contraseñas reales que permiten avanzar. Por otro —y más importante para el informe— **medir la resistencia real** de las contraseñas de la organización: cuántas cayeron, en cuánto tiempo, y con qué patrones, es una métrica de riesgo mucho más elocuente que "la política exige ocho caracteres". Y la frontera ética es la de siempre: se crackean **solo** hashes propios o del alcance autorizado ([Clase 025](#)); hacerlo sobre credenciales ajenas es delito.

Definiciones y características

- **Tipo/modo de hash:** identificador del algoritmo criptográfico usado para generar el hash (NTLM, MD5, SHA-256, bcrypt, sha512crypt). Característica clave: en Hashcat corresponde al parámetro numérico `-m` (por ejemplo, `1000` para NTLM); en John, al parámetro `--format`.
- **Enfoque defensivo:** conocer qué modo de hash usa cada sistema propio permite a seguridad estimar de antemano cuán rápido podría un atacante crackear una filtración de esas credenciales.
- **Ataque de diccionario:** prueba cada palabra de una lista predefinida contra el hash objetivo. Característica clave: extremadamente rápido y efectivo, porque la mayoría de contraseñas humanas derivan de palabras reales o patrones comunes.
- **Enfoque defensivo:** correr el mismo ataque contra los propios hashes de usuarios (con autorización) identifica contraseñas débiles antes de que lo haga un atacante externo.
- **Rainbow table:** estructura precomputada que relaciona hashes con sus contraseñas de origen para acelerar la búsqueda inversa sin salt. Característica clave: es completamente ineficaz contra hashes correctamente salados, porque cada salt distinto invalida la tabla precomputada.
- **Enfoque defensivo:** exigir salt individual por credencial en cualquier sistema propio es un requisito no negociable de cualquier revisión de almacenamiento de contraseñas.
- **Reglas de mutación (rules):** conjunto de transformaciones (mayúscula inicial, sufijo numérico, sustitución leet) aplicadas automáticamente a cada palabra de un diccionario. Característica clave: multiplican exponencialmente la cobertura efectiva de una wordlist sin aumentar su tamaño en disco.
- **Enfoque defensivo:** una política de contraseñas que solo exige mayúscula+número+símbolo no detiene reglas como `best64`, porque anticipan exactamente ese patrón; la longitud importa más que la complejidad forzada.
- **Ataque de máscara:** fuerza bruta dirigida por un patrón de caracteres conocido o supuesto, como `u?l?l?l?l?d?d?d?d` (mayúscula, cuatro minúsculas, cuatro dígitos). Característica clave: reduce drásticamente el espacio de búsqueda frente a una fuerza bruta sin restricciones.
- **Enfoque defensivo:** evitar patrones predecibles como "Nombre+Año" en políticas corporativas reduce la efectividad de máscaras dirigidas contra la organización.
- **Salt:** valor aleatorio único añadido a cada contraseña antes de aplicar la función hash. Característica clave: obliga al atacante a crackear cada hash de forma individual, eliminando la ventaja de precomputar resultados una sola vez.

- *Enfoque defensivo*: verificar que el framework o base de datos usado internamente genere salt automáticamente (bcrypt y Argon2 lo hacen por diseño) es parte de cualquier auditoría de autenticación.
- **KDF lento (bcrypt, scrypt, Argon2)**: función de derivación de clave diseñada deliberadamente para ser computacionalmente costosa, con un factor de coste ajustable. Característica clave: hace que probar millones de contraseñas por segundo sea económicamente inviable, a diferencia de hashes rápidos como MD5 o NTLM.
- *Enfoque defensivo*: migrar sistemas legados de MD5/SHA1 a bcrypt o Argon2, y ajustar el factor de coste según el hardware disponible, es una de las mejoras de seguridad con mayor retorno frente al cracking masivo.

Glosario

Término	Definición concisa
Cracking	Recuperar la contraseña en claro a partir de su hash
Identificación de hash	Determinar el algoritmo antes de atacar
John the Ripper / Hashcat	Herramientas de cracking (CPU y GPU)
Ataque de diccionario	Probar listas de contraseñas conocidas
rockyou	Diccionario clásico de contraseñas filtradas
Reglas de mutación	Transforman el diccionario imitando hábitos humanos
Ataque de máscara	Fuerza bruta dirigida por un patrón conocido
Ataque híbrido	Combina diccionario y máscara
Aceleración por GPU	Miles de millones de candidatos por segundo
Hash rápido	MD5, NTLM, SHA-*; vulnerable al cracking a escala
KDF lento	bcrypt, scrypt, Argon2; diseñado para resistir el cracking
Salt	Valor único por hash que inutiliza las rainbow tables
Rainbow table	Tabla precomputada; el salt la inutiliza
Métrica de resistencia	Cuántas contraseñas caen y en cuánto tiempo

Herramientas y preparación

- **John the Ripper**: `sudo apt install john`; ofrece autodetección de formato y es especialmente cómodo en CPU.
- **Hashcat**: `sudo apt install hashcat`; domina en velocidad cuando hay GPU disponible. Verifica los dispositivos con `hashcat -I`.
- **rockyou.txt**: wordlist estándar incluida en Kali, descomprimir con `gunzip -k /usr/share/wordlists/rockyou.txt.gz`.
- **Reglas**: `best64.rule` incluida por defecto en `/usr/share/hashcat/rules/`; también existen conjuntos más agresivos como `rockyou-30000.rule`.
- **hashid** o `hashcat --identify`: para determinar el tipo de hash antes de lanzar cualquier ataque.

- Hashes obtenidos exclusivamente en tu propio laboratorio (por ejemplo, con `secretsdump.py` de una VM propia, o hashes generados manualmente para práctica con `openssl passwd` o `mkpasswd`).
- **GPU con drivers actualizados** (opcional pero recomendado): Hashcat aprovecha CUDA/OpenCL; sin GPU, funciona igualmente en modo CPU (`-D 1`) aunque más lento.
- **hcxtools**: conjunto de utilidades para convertir capturas de tráfico (por ejemplo handshakes WPA) al formato que Hashcat necesita, útil para prácticas de laboratorio con redes Wi-Fi propias.

Laboratorio guiado

El cracking de contraseñas se practica exclusivamente sobre hashes propios o extraídos en un engagement autorizado por escrito. Las contraseñas recuperadas son datos altamente sensibles: se documentan únicamente para demostrar impacto y deben destruirse al cerrar el laboratorio o el contrato, nunca reutilizarse fuera del alcance acordado.

1. Identifica el tipo de hash antes de cualquier ataque: `hashcat --identify hashes.txt` o `hashid '$1$xyz$abc ... '`.
2. Lanza un ataque de diccionario puro con Hashcat (ejemplo para NTLM, modo `1000`): `hashcat -m 1000 -a 0 hashes.txt /usr/share/wordlists/rockyou.txt`.
3. Repite el ataque añadiendo reglas de mutación: `hashcat -m 1000 -a 0 hashes.txt rockyou.txt -r /usr/share/hashcat/rules/best64.rule`.
4. Ejecuta un ataque de máscara para contraseñas de 8 caracteres con patrón letra+dígito: `hashcat -m 1000 -a 3 hashes.txt '?l?l?l?l?d?d?d?d'`.
5. Prueba un ataque híbrido diccionario más máscara (por ejemplo, palabra + año): `hashcat -m 1000 -a 6 hashes.txt rockyou.txt '?d?d?d?d'`.
6. Repite el proceso con John the Ripper, que autodetecta el formato: `john --wordlist=rockyou.txt hashes.txt` y consulta resultados con `john --show hashes.txt`.
7. Verifica los resultados acumulados en Hashcat: `hashcat -m 1000 hashes.txt --show`.
8. Compara la velocidad reportada por Hashcat en CPU (`-D 1`) frente a GPU (`-D 2`) si tu laboratorio dispone de una GPU dedicada.
9. Genera un hash bcrypt de práctica con `mkpasswd -m bcrypt` y compara el tiempo de un solo intento de crackeo frente a un hash NTLM equivalente.
10. Si dispones de una captura de laboratorio propio, convierte un handshake WPA con `hcxpcapngtool` al formato Hashcat (`-m 22000`) y ejecuta un ataque de diccionario sobre él.
11. Clasifica cada contraseña recuperada según su debilidad (palabra de diccionario simple, patrón año+símbolo, reutilización evidente) y redacta una recomendación de política.

Ejercicios

1. Identifica tres tipos de hash distintos (por ejemplo NTLM, MD5 y bcrypt) y anota su modo `-m` correspondiente en Hashcat.
2. Compara la tasa de éxito de un ataque de diccionario puro frente a diccionario más regla `best64` sobre el mismo conjunto de hashes.
3. Diseña una máscara Hashcat para contraseñas del patrón `Nombre2024!` y ejecútala.
4. Ejecuta un ataque híbrido (`-a 6` o `-a 7`) combinando diccionario y máscara.
5. Mide aproximadamente la diferencia de velocidad (hashes por segundo) entre CPU y GPU para el mismo modo de hash.

6. Explica por qué un hash bcrypt con factor de coste 12 cambia radicalmente la estrategia de ataque frente a un NTLM.

Reto verificable

Reto: a partir de un conjunto de hashes de tu laboratorio (mínimo 10, con al menos dos tipos distintos), crackea el máximo número posible y produce un informe de fortaleza de contraseñas.

Criterio de aceptación: el alumno documenta el tipo de hash de cada conjunto, el ataque exacto usado para crackearlo, el porcentaje total recuperado, una clasificación de las contraseñas obtenidas por nivel de debilidad, y una recomendación concreta de política de contraseñas (longitud mínima, uso de MFA, prohibición de patrones comunes).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Hashcat reporta "No hashes loaded"	El modo <code>-m</code> no coincide con el formato real del hash; confirmar con <code>hashcat --identify</code>
Hashcat reporta "Separator unmatched"	El archivo mezcla formatos (usuario:hash y hash suelto); limpiar el archivo dejando un formato consistente
Velocidad de cracking muy baja e inesperada	El hash usa un KDF lento por diseño (bcrypt/Argon2) o no hay GPU disponible; ajustar expectativas de tiempo
John no detecta el formato automáticamente	Especificar el formato manualmente con <code>--format=nt</code> o el que corresponda
<code>rockyou.txt</code> no se encuentra	Falta descomprimir <code>/usr/share/wordlists/rockyou.txt.gz</code> con <code>gunzip -k</code> primero
Ataque de máscara tarda demasiado	El patrón elegido es demasiado amplio; reducir el espacio con suposiciones más específicas sobre longitud y conjunto de caracteres

Preguntas frecuentes

 **¿Debo usar John the Ripper o Hashcat?** John destaca en CPU, en la variedad de formatos soportados y en su autodetección cómoda. Hashcat domina en velocidad bruta cuando hay GPU disponible. En la práctica profesional se usan ambos según el hash y el hardware disponible.

 **¿Por qué no logro crackear un hash bcrypt o Argon2?** Porque están diseñados deliberadamente para ser lentos y usar salt individual; cada intento de contraseña cuesta un cómputo significativo, a diferencia de MD5 o NTLM que permiten miles de millones de intentos por segundo. Esa lentitud es precisamente su mecanismo de defensa.

 **¿Realmente ayudan tanto las reglas de mutación?** Sí, de forma considerable: la mayoría de contraseñas humanas son una palabra base con una mutación predecible (mayúscula inicial, año al final, símbolo de cierre). Reglas como `best64` capturan exactamente esos patrones observados en filtraciones reales.

? Qué debo hacer con las contraseñas que logro crackear en un engagement real? Se documentan únicamente para demostrar impacto (por ejemplo, reutilización de credenciales o política débil) en el informe entregado al cliente, y se destruyen de forma segura al cierre del compromiso, conforme a lo pactado en las reglas de engagement.

? ¿Por qué la longitud importa más que la complejidad en una política de contraseñas? Porque cada carácter adicional multiplica exponencialmente el espacio de búsqueda, mientras que exigir mayúsculas/símbolos suele producir patrones predecibles (mayúscula inicial, símbolo final) que las reglas de mutación ya anticipan; NIST SP 800-63B refleja este cambio de enfoque.

Referencias

- Hashcat, documentación oficial — <https://hashcat.net/hashcat/>
- John the Ripper (Openwall) — <https://www.openwall.com/john/>
- NIST SP 800-63B, Digital Identity Guidelines - Authentication and Lifecycle Management — <https://pages.nist.gov/800-63-3/sp800-63b.html>
- OWASP, Password Storage Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- MITRE ATT&CK, T1110 Brute Force — <https://attack.mitre.org/techniques/T1110/>

Material descargable

-  [Guía en PDF](#) — versión imprimible de esta clase.
-  [Presentación \(PPTX\)](#) — deck para proyectar en clase.

Clase anterior

[Clase 079 — Pivoting y reenvío de puertos](#)

Siguiente clase

[Clase 081 — Ataques a credenciales: fuerza bruta y password spraying](#)